

VISIONQUEUE

Technical Design Document — V2.0

Intelligent Real-Time Queue and Crowd Management System

Research-aligned implementation blueprint

Research baseline checked: 20 August 2026

Primary product source: VisionQueue Master PRD

Scope: P0 + conditional Core+; P2 and Advanced are explicitly outside V1

Property	TDD baseline
Deployment	Windows local computer; single camera
Processing	Local CV; optional NVIDIA GPU acceleration; CPU fallback
Primary counting signal	Person detection + tracking
College CV requirement	YuNet face detection runs alongside person detection
Application boundary	FastAPI + WebSocket
Persistence	SQLite with WAL; asynchronous writes
UI	React 19.2 + TypeScript + Vite 7 + Tailwind CSS 4
Testing	pytest + HTTPX/TestClient; Vitest + React Testing Library; repeatable video fixtures
Architecture style	Modular single application, not microservices

Source alignment: The Master PRD explicitly says it supersedes the earlier PRDs, is the authoritative product specification, and intentionally leaves architecture and technology choices to the TDD. [filecite?turn9file5?L325-L335?](#)

Research alignment: Current official documentation was checked for YOLO26/export/licensing, ONNX Runtime CUDA compatibility, YuNet/OpenCV, ByteTrack/trackers, FastAPI/WebSockets, React/Vite, Tailwind, Recharts, TanStack Query and SQLite WAL.

Contents

1. 1. Document Control and Interpretation
2. 2. Product-to-Technical Scope Boundary
3. 3. System Context
4. 4. V1 Functional Architecture
5. 5. Architecture Principles
6. 6. Constraints and Assumptions
7. 7. V1 Technology Stack
8. 8. Runtime and Version Baseline
9. 9. Technology Evaluation and Rationale
10. 10. System Architecture
11. 11. Runtime Processes and Concurrency
12. 12. Camera/Input Subsystem
13. 13. Frame Scheduling and Freshness
14. 14. Person Detection
15. 15. Face Detection
16. 16. Tracking
17. 17. Counting and Crossing
18. 18. Session and Approximate Unique Counting
19. 19. Occupancy and Crowd Analytics
20. 20. Reliability State Machine
21. 21. Alert Engine
22. 22. Live State Model
23. 23. Backend/API
24. 24. WebSocket

24.1 Required Runtime Sequence Flows

The following flows make the implementation boundary explicit. They are intentionally application-level and do not expose model-specific internals to the frontend.

A. Normal frame processing

```
sequenceDiagram
    participant Cam as CameraSource
    participant Buf as LatestFrameBuffer
    participant Det as PersonDetector
    participant Face as FaceDetector
    participant Trk as ByteTrack
    participant Cnt as Counter
    participant Ana as Analytics
    participant State as LiveState
    participant Pub as Publisher

    Cam->>Buf: frame + capture timestamp
    Buf->>Det: newest frame
    Buf->>Face: same/newest frame at validated face cadence
    Det-->>Trk: person detections
    Trk-->>Cnt: active tracks
    Cnt->>Ana: count + crossing events
    Ana->>State: occupancy + crowd
    Face-->>State: face detection status
    State->>Pub: publish current snapshot
```

B. Person detection → tracking → counting

```
sequenceDiagram
    participant D as PersonDetector
    participant T as ByteTrack
    participant X as ROI/Crossing
    participant C as CountEngine
    participant S as SessionRegistry

    D->>T: detections
```

```

T->>X: track ID + bbox
X->>X: calculate bottom-center
X->>X: evaluate ROI/boundary
X->>C: entry/exit/current membership
C->>S: first-in-session track?
S-->>C: new/seen
C-->>C: update current + approximate unique count

```

C. Live dashboard update

```

sequenceDiagram
    participant CV as CV/Domain
    participant WS as WebSocketHub
    participant UI as React Dashboard

    CV->>WS: new LiveState
    WS->>UI: state_update
    UI->>UI: validate schema_version
    UI->>UI: update live widgets
    UI->>UI: update status/alerts

```

D. Camera failure and recovery

```

sequenceDiagram
    participant Cam as CameraSource
    participant Rel as ReliabilityMonitor
    participant State as StateManager
    participant UI as Dashboard

    Cam-->>Rel: read failure
    Rel->>State: camera unavailable
    State->>UI: OFFLINE + last reliable values
    Rel->>Cam: reconnect attempts
    Cam-->>Rel: stream recovered
    Rel->>State: enter STARTING
    State->>UI: STARTING
    Rel->>Rel: confidence/performance check
    Rel->>State: healthy
    State->>UI: LIVE

```

E. GPU inference failure → CPU fallback

```

sequenceDiagram
    participant RT as InferenceRuntime
    participant GPU as CUDA EP
    participant CPU as CPU EP
    participant Rel as ReliabilityMonitor
    participant State as StateManager

    RT->>GPU: infer(frame)
    GPU-->>RT: provider/runtime error
    RT->>RT: record fallback event
    RT->>CPU: infer(frame)
    CPU-->>RT: detections
    RT->>Rel: CPU performance measurement
    Rel->>State: LIVE or DEGRADED according to validated threshold

```

F. Session start/stop

```

sequenceDiagram
    participant UI as Dashboard
    participant API as FastAPI
    participant S as SessionManager
    participant DB as SQLite

    UI->>API: POST /api/v1/session/start
    API->>S: start()
    S->>DB: create session record
    S-->>API: session metadata
    API-->>UI: session response
    UI->>API: POST /api/v1/session/stop
    API->>S: stop()
    S->>DB: close session record

```

```
S-->>API: final metadata
API-->>UI: stopped response
```

G. Historical data retrieval

```
sequenceDiagram
    participant UI as React
    participant Q as TanStack Query
    participant API as FastAPI
    participant DB as SQLite

    UI->>Q: request history
    Q->>API: GET /api/v1/history
    API->>DB: indexed query
    DB-->>API: measurements
    API-->>Q: MeasurementPage
    Q-->>UI: cached result
    UI->>UI: transform for Recharts
```

H. Frontend reconnect

```
sequenceDiagram
    participant UI as WebSocketClient
    participant WS as FastAPI WebSocket
    participant State as LiveStateStore

    UI->>WS: connect
    WS->>State: read current snapshot
    State-->>WS: LiveState
    WS-->>UI: state_snapshot
    UI->>UI: replace stale local live state
    WS-->>UI: future state_update messages
```

I. Mock-data development flow

```
sequenceDiagram
    participant Mock as MockVisionStateSource
    participant API as FastAPI
    participant WS as WebSocket
    participant UI as React
    participant DB as SQLite

    Mock->>API: LiveState
    API->>WS: publish state
    WS->>UI: state_snapshot/update
    Mock->>DB: simulated measurement/event
    UI->>API: history request
    API->>DB: query simulated history
    DB-->>API: history
    API-->>UI: history
```

25. 25. Video Presentation Boundary
26. 26. Database
27. 27. Frontend
28. 28. Mock and Simulation
29. 29. Data Contracts
30. 30. API Contracts
31. 31. Database Schema
32. 32. Repository Structure
33. 33. Configuration
34. 34. Deployment
35. 35. Logging and Observability
36. 36. Security and Privacy
37. 37. Testing Strategy
38. 38. Performance Engineering
39. 39. Failure Injection and Recovery
40. 40. Technical Risks
41. 41. Architecture Rationale Records
42. 42. Known Limitations

- 43. 43. Future / Advanced Architecture
- 44. 44. Validation Plan
- 45. 45. Final Architecture Review
- 46. 46. Technical Decisions Requiring Validation
- 47. 47. PRD ↔ TDD Consistency Check
- 48. 48. References

1. Document Control and Interpretation

Item	Definition
Document type	Technical Design Document; implementation blueprint, not a college essay
Authority	VisionQueue Master PRD for product behavior; this TDD for technical implementation
Version	2.0
Research date	20 August 2026
Measurement policy	No measured FPS/latency/accuracy/cost is claimed unless produced by the project's own test runs
Scope policy	P0 is mandatory; Core+ is conditional; P2/Advanced are not V1 dependencies
Terminology	"Must" = implementation requirement; "should" = strong engineering guidance; "TBD" = unresolved technical parameter

The Master PRD states that it defines what the system must do and leaves libraries, frameworks, architecture, literal database schema, APIs, module interfaces and testing strategy to the TDD. [filecite?turn9file5?L329-L335?](#)

This document therefore does not rewrite product requirements. Where the PRD leaves a parameter open, the TDD defines the mechanism and the validation method rather than inventing a final number.

2. Product-to-Technical Scope Boundary

2.1 P0 — implementation scope

- Live feed from laptop webcam or phone-as-webcam.
- Person detection and face detection together.
- One configurable rectangular ROI.
- Person tracking across frames.
- Current count and approximate running unique-people-this-session count.
- Entry/exit boundary crossing.
- Capacity-based occupancy percentage.
- LOW/MODERATE/HIGH/CRITICAL crowd level with debounce.
- Live dashboard with feed, counts, occupancy, crowd level, confidence/camera state.
- Exactly two P0 alert categories: critical occupancy and camera/detection failure.
- Explicit STARTING/LIVE/DEGRADED/UNSTABLE/OFFLINE states.
- Local-only processing by default; no biometric identification.

These items are directly supported by the Master PRD's P0 scope. [filecite?turn9file6?L405-L422?](#)

2.2 Core+ — only after P0

- Crowd trend.
- Peak crowd and peak occupancy.
- Historical logging.
- Basic historical analytics/charting.

Core+ must not degrade P0. The PRD explicitly says Core+ is optional and is cut if it threatens the P0 deadline.

[filecite?turn9file1?L76-L82?](#)

2.3 Deferred

- Waiting-time estimation.
- Queue movement status.
- Full alert taxonomy beyond the two P0 alerts.
- Snapshot/event recording as a product feature.

These are P2/deferred rather than hidden technical gaps. [filecite?turn9file7?L433-L444?](#)

3. System Context

flowchart LR
OP[Operator] --> UI[VisionQueue Dashboard]
CAM[Webcam / Phone-as-Webcam] --> APP[VisionQueue Local Application]
TEST[Video File
development only] --> APP
APP --> UI
APP --> DB[(SQLite WAL)]
CFG[Configuration] --> APP

The V1 system is intentionally local. The phone, if used, is a camera only; all computer-vision work and dashboard serving remain on the laptop. The PRD notes that Wi-Fi phone streaming adds latency and depends on Wi-Fi stability, while the built-in webcam is the simpler default. [filecite?turn9file5?L347-L351](#)

4. V1 Functional Architecture

flowchart TD
A[Camera Frame] --> B[Capture Timestamp]
B --> C[Latest-Frame Buffer]
C --> D[Person Detection]
C --> E[Face Detection]
D --> F[Detection Filtering]
F --> G[ByteTrack]
G --> H[ROI Membership]
H --> I[Boundary Crossing]
I --> J[Current Count]
G --> K[Session Track Registry]
K --> L[Approx. Unique Count]
J --> M[Occupancy]
M --> N[Crowd Level]
J --> O[Alert Engine]
N --> O
C --> P[Freshness/Performance Monitor]
P --> Q[Reliability State]
J --> S[Live State]
L --> S
M --> S
N --> S
O --> S
Q --> S
E --> S
S --> API[FastAPI]
API --> WS[WebSocket]
WS --> UI[React]
S --> PW[Persistence Queue]
PW --> DB[(SQLite)]

4.1 Key separation

- Detection answers: where are people/faces in this frame?
- Tracking answers: which detections are likely the same short-lived track across adjacent frames?
- Counting answers: which active tracks currently matter to the configured ROI/boundary?
- Analytics answers: what does the count mean operationally?
- State management answers: can the application currently trust and publish these values?
- Backend answers: how are application facts exposed to clients?
- Frontend answers: how are those facts presented to an operator?

5. Architecture Principles

Principle	Application to VisionQueue
Freshness over completeness	Drop stale frames rather than process an ever-growing backlog.
Failure is data	Camera/runtime/processing failures become explicit state, never an invented zero.
CV isolation	Model and tracker internals do not leak into REST/UI contracts.
Single source of live truth	One LiveState object is authoritative for dashboard values.
Asynchronous persistence	SQLite writes are downstream of the critical CV loop.

Replaceable components	Camera, detector, tracker and state producer are interfaces.
Session-scoped identity	Track IDs are temporary technical identifiers, not human identity.
P0 before Core+	History and charts are allowed only after live reliability is proven.
No unnecessary infrastructure	One local process plus SQLite is enough for V1.
Measured performance	Targets are test criteria, never claims about actual hardware.

6. Constraints and Assumptions

Constraint / assumption	Technical consequence
~20-day student build	Prefer a modular monolith over microservices.
Zero/minimal budget	No cloud GPU, managed database or paid API.
No dedicated GPU assumed	CPU path must exist and non-CV work must not require GPU.
Optional GPU	CUDA is a capability of the inference layer, not the whole system.
Single camera	No distributed synchronization or multi-camera identity model.
Laptop local processing	Low operational overhead and localhost-first security.
Pilot likely campus queue/shop counter	Camera placement can be controlled; moving-camera compensation is not a V1 requirement.
Real footage required	Test fixtures must include actual pilot footage.

The PRD explicitly says to treat the laptop as CPU-only until a GPU is confirmed, while retaining GPU-related feasibility as an open implementation consideration. [filecite?turn9file9?L556-L563](#)

7. V1 Technology Stack

Layer	V1 technology	Technical role	License / compatibility note
Application language	Python 3.11.x	CV, analytics, backend	Chosen to stay within current Ultralytics/FastAPI compatibility range; exact patch pinned in lockfile.
CV framework	OpenCV 5.0.x	Capture, image ops, visualization, YuNet API	Apache 2.0; current Python wheels expose OpenCV 5.0.0.93; YuNet's current model notes target OpenCV 5.x.
Person detector	Ultralytics YOLO26n	Person detection	Ultralytics documents AGPL-3.0 / Enterprise licensing; commercial use needs separate review.
Person inference runtime	ONNX Runtime 1.29.x	ONNX model execution	CPU and CUDA packages; exact compatible package pinned.
GPU execution	ONNX Runtime CUDA EP	NVIDIA GPU acceleration	CUDA/cuDNN compatibility must match ORT package.
CPU execution	ONNX Runtime CPU EP	Universal fallback	Used when CUDA is unavailable/unhealthy or CPU-only development is required.
Face detector	YuNet ONNX via OpenCV FaceDetectorYN	Face detection only	No face recognition; OpenCV Zoo provides YuNet implementation/model.
Tracker	ByteTrack	Short-term multi-person association	No appearance ReID; suitable for fixed-camera V1 baseline.
Backend	FastAPI 0.141.x + Uvicorn	REST, validation, WebSocket host	FastAPI supports OpenAPI, WebSockets, testing and async endpoints.
Live transport	WebSocket	Low-volume live state	Not video transport.
Database	SQLite 3.x + WAL	Local configuration/history/events	No DB server; WAL supports reader/writer concurrency suitable for V1.
Frontend	React 19.2.x	Dashboard UI	Current React major baseline.
Frontend language	TypeScript 5.9.x	Static typing	Pinned for reproducible frontend builds.
Build/dev server	Vite 7.x	SPA dev server/build	Current Vite requires Node 20.19+ or 22.12+.
Styling	Tailwind CSS 4.3.x	UI styling	Current v4 line; Vite plugin integration supported.
Charts	Recharts 3.x	History/trend charts	MIT; React-native component model.
Server-state	TanStack Query 5.x	REST/history caching and mutations	WebSocket live state remains separate.
Backend tests	pytest + FastAPI TestClient/HTTPX	Unit/API/WebSocket tests	FastAPI documents TestClient WebSocket testing.

Frontend tests	Vitest + React Testing Library	Component/UI tests	Lightweight SPA testing stack.
Version control	Git + GitHub	Source control, review, CI	No runtime dependency.

The stack is now presented as an explicit V1 baseline rather than as a list of unresolved technology decisions. Exact package patch versions should be locked after the first clean reference-environment installation; the TDD avoids pretending that a package version is stable forever.

8. Runtime and Version Baseline

Runtime	Baseline
Python	3.11.x
Node.js	22.12+ LTS-compatible baseline for Vite 7
React	19.2.x
TypeScript	5.9.x
Vite	7.x
Tailwind	4.3.x
FastAPI	0.141.x
ONNX Runtime	1.29.x; CUDA package only on GPU reference environment
OpenCV	5.0.x
YOLO26	YOLO26n
YuNet	Current OpenCV Zoo YuNet ONNX model; use model variant compatible with selected OpenCV DNN backend
SQLite	System/runtime SQLite 3.x with WAL enabled

Research basis: React currently documents 19.2 as the latest major version; Vite 7 requires Node 20.19+ or 22.12+; FastAPI's July 2026 release notes list 0.141.1; ONNX Runtime lists 1.29.0 as current release; OpenCV Python has a 5.0.0.93 release; Tailwind's current line is 4.3. [\[cite?turn4search5?turn3search0?turn3search6?turn6search7?turn6search0?turn4search1?\]](#)

Version policy: do not blindly upgrade during V1. Pin the exact working patch versions in the Python and Node lockfiles. Upgrade only after the regression suite passes.

9. Technology Evaluation and Rationale

9.1 OpenCV 5.x

OpenCV is used for camera I/O, frame manipulation, drawing, ROI operations and YuNet integration. It avoids introducing a second camera abstraction. The current Python package line includes OpenCV 5.0.0.93 and is Apache 2.0. YuNet's current OpenCV Zoo documentation specifically describes a dynamic-input model compatible with OpenCV 5.x's ONNX Runtime engine.

[\[cite?turn6search0?turn0search2?\]](#)

Alternative: OpenCV 4.x. It remains viable, but the current YuNet model documentation has explicit OpenCV 5.x support, making OpenCV 5.x the cleaner current baseline. Validation: confirm `FaceDetectorYN` behavior and model loading on the actual Windows Python environment.

9.2 YOLO26n

YOLO26n is the smallest YOLO26 detection scale and is documented for real-time detection and ONNX export. Ultralytics documents ONNX export and gives the end-to-end YOLO26 ONNX output shape as detections containing box coordinates, confidence and class ID. [\[cite?turn0search0?turn7search0?\]](#)

Alternative: YOLO11n. It remains a reasonable fallback if YOLO26n export/runtime integration or local accuracy is poor. The V1 design therefore isolates the detector behind an interface so changing the model does not change counting, analytics or API contracts.

License: Ultralytics documents AGPL-3.0 and an Enterprise option. The college V1 is educational/non-commercial, but the project must retain license notices and re-evaluate licensing before commercial/internal production distribution.

[\[cite?turn0search1?turn0search14?\]](#)

9.3 ONNX Runtime

ONNX Runtime is the deployment runtime because it gives the application a model-format boundary between model export and inference. The official Python installation documentation provides CPU and CUDA GPU packages, while the CUDA provider documentation lists CUDA/cuDNN compatibility by ORT version. [\[cite?turn3search1?turn1search2?\]](#)

Important correction from the earlier TDD: the project should not install both ``onnxruntime`` and ``onnxruntime-gpu`` in the same Python environment. The official documentation says only one package should be installed in an environment. GPU environments use ``onnxruntime-gpu``; CPU-only environments use ``onnxruntime``. [\[cite?turn3search1?\]](#)

9.4 CUDA versus TensorRT

CUDA EP remains the V1 GPU path because it preserves one runtime abstraction and is easier to keep compatible with CPU fallback. TensorRT can be faster on NVIDIA hardware, and Ultralytics documents TensorRT export, but introducing TensorRT adds another runtime/toolchain and is not required unless CUDA EP fails the measured performance target. [\[cite?turn7search0?\]](#)

9.5 ByteTrack

ByteTrack is the V1 tracker because the camera is expected to be fixed and V1 does not require appearance-based identity. Current Ultralytics tracking documentation describes ByteTrack as the fastest/simple baseline without ReID or camera-motion compensation, while BoT-SORT adds camera-motion compensation and optional ReID. [\[cite?turn7search1?\]](#)

Alternative: BoT-SORT. It becomes relevant if the pilot shows camera movement or severe ID switching. It is not used merely because it has more features.

9.6 YuNet

YuNet is retained because the college requirement is face detection, not identity. OpenCV Zoo describes it as lightweight and provides an OpenCV ``FaceDetectorYN`` implementation. [\[cite?turn0search2?turn0search5?\]](#)

9.7 FastAPI + Uvicorn

FastAPI provides REST, OpenAPI validation, WebSockets and test tooling. The official documentation exposes WebSocket routes and WebSocket testing through TestClient. Uvicorn is the ASGI server used to run the application locally.

[\[cite?turn2search5?turn2search11?turn1search3?\]](#)

9.8 SQLite + WAL

SQLite is appropriate because V1 is one local application with small aggregated history. WAL provides a reader-friendly transaction model, but the architecture still uses one serialized persistence worker so database activity cannot become a source of frame-loop contention. SQLite documents that active WAL state includes the main DB, ``-wal``, and ``-shm`` files; backups/copies must preserve the active state correctly. [\[cite?turn0search12?turn0search8?\]](#)

9.9 React 19.2 + Vite 7

React remains appropriate for a local operator SPA. React's current docs list 19.2 as latest and explicitly support building from scratch with Vite; Vite provides a React TypeScript template and current fast development/build workflow.

[\[cite?turn4search5?turn2search8?turn2search1?\]](#)

9.10 Tailwind 4.3

Tailwind 4.x is the current line and provides a Vite plugin. This is a direct fit for a local dashboard and avoids adding a component framework that would dictate the UI design. [\[cite?turn2search0?turn4search1?\]](#)

9.11 Recharts + TanStack Query

Recharts is a composable React chart library and is sufficient for count/occupancy trend charts. TanStack Query handles asynchronous REST server state and history caching; it should not be used as the WebSocket live-state store.

[\[cite?turn1search4?turn1search5?\]](#)

10. System Architecture

flowchart TB

subgraph INPUT["INPUT"]

```

    SRC[CameraSource]
    BUF[LatestFrameBuffer]
end
subgraph CV["VISION"]
    PRE[Preprocessor]
    PD[PersonDetector]
    FD[FaceDetector]
    TR[Tracker]
end
subgraph DOMAIN["DOMAIN"]
    ROI[ROI / Crossing]
    CNT[Counting]
    ANA[Occupancy / Crowd]
    ALT[Alerts]
    RS[Reliability Monitor]
    LS[LiveState]
end
subgraph APP["APPLICATION"]
    API[REST API]
    WS[WebSocket Hub]
    PQ[Persistence Queue]
end
DB[(SQLite WAL)]
UI[React Dashboard]

SRC --> BUF
BUF --> PRE
PRE --> PD
PRE --> FD
PD --> TR
TR --> ROI
ROI --> CNT
CNT --> ANA
CNT --> ALT
BUF --> RS
PD --> RS
TR --> RS
ANA --> RS
FD --> LS
CNT --> LS
ANA --> LS
ALT --> LS
RS --> LS
LS --> API
LS --> WS
LS --> PQ
PQ --> DB
WS --> UI
API --> UI

```

10.1 Why this is a modular monolith

All modules run locally and can communicate in-process. This provides independent code boundaries without the operational cost of separate services, service discovery, message brokers or distributed deployment.

11. Runtime Processes and Concurrency

Execution unit	Work	Priority
Capture loop	Read camera; timestamp; publish newest frame	High
Vision loop	Person/face inference, tracking	Highest
Domain loop	Counting, analytics, state transition	Highest
Persistence worker	Batch/periodic SQLite writes	Low
WebSocket publisher	Broadcast current state	Medium
FastAPI request handlers	Configuration/history/health	Medium/Low
Frontend browser	Render state/history	Separate process

A single Python process can host the capture/CV/domain loops using dedicated threads or a controlled worker model. The exact threading model should be selected after profiling. Heavy synchronous inference must not run inside an async FastAPI request handler.

11.1 Backpressure

Only the newest frame is valuable for live monitoring. Persistence and WebSocket queues must also be bounded. If a client cannot keep up, drop intermediate state updates and send the next full snapshot; never block the CV loop waiting for a slow browser.

12. Camera/Input Subsystem

```
class CameraSource:
    open() -> CameraStatus
    read() -> FramePacket | None
    reconnect() -> CameraStatus
    close() -> None
    metadata() -> CameraMetadata
```

```
FramePacket:
    frame
    capture_timestamp
    source_sequence
```

Source	V1 role	Failure behavior
Laptop webcam	Primary deployment source	OFFLINE -> reconnect -> STARTING
Phone-as-webcam	Supported deployment source	Source/network failure -> OFFLINE
Video file	Development/repeatability fixture	End-of-file is a controlled test condition

OpenCV VideoCapture supports camera and video-file capture, making it a suitable common abstraction for these sources. [R1-style OpenCV source; see references.]

13. Frame Scheduling and Freshness

```
Capture producer
|
v
[bounded latest-frame slot]
|
+--> replace old frame when new frame arrives
|
v
Vision consumer
|
+--> if frame age > freshness budget: discard
|
v
inference -> tracking -> state
```

13.1 Frame age

Every frame carries a capture timestamp. The system records frame age at the beginning of inference and before publication. A frame can be processed successfully but still be operationally stale; stale state must therefore be detected separately from inference failure.

13.2 No unbounded queues

- Camera queue: capacity 1–2, newest frame wins.
- Persistence queue: bounded; non-critical measurements may be coalesced.
- WebSocket queue: per-client bounded; slow clients receive the newest snapshot.
- Logs: rate-limited for high-frequency metrics.

14. Person Detection

```
Detection {
    bbox: [x1, y1, x2, y2]
    confidence: float
    class_id: int
    frame_timestamp
```

```

}

PersonDetector {
  infer(frame) -> list[Detection]
}

```

YOLO26n is exported to ONNX. Ultralytics documents YOLO26 ONNX export and supports end-to-end outputs with `[x1, y1, x2, confidence, class\_id]`-style detection records. [?cite?turn7search0?](#)

14.1 Detection frequency

The detector does not have to run on every camera frame. A validated detection cadence may be used while the tracker maintains continuity. The cadence is a performance parameter, not a fixed requirement.

14.2 Detection filtering

- Keep only the person class for counting.
- Apply a configurable confidence threshold.
- Reject malformed/out-of-frame boxes.
- Do not count face detections as people.

15. Face Detection

YuNet runs as a separate face-detection pipeline. It is not part of the person-counting decision.

```

Camera frame
  |
  +--> Person detector ----> tracking/counting
  |
  +--> YuNet face detector -> face boxes/overlay/status

```

Because face detection satisfies the college requirement but does not drive product counting, it may run at a lower validated cadence than person detection if required to preserve real-time behavior. The PRD requires both to run and be visible in the demo but does not specify equal per-frame frequency. [?filecite?turn9file5?L355-L367?](#)

15.1 Privacy boundary

- No face embeddings.
- No names.
- No persistent face IDs.
- No face database.
- No cross-session face identity.

16. Tracking

16.1 Tracker contract

```

Track {
  track_id: int
  bbox
  confidence
  state: ACTIVE | LOST | ENDED
  first_seen
  last_seen
}

```

```

Tracker.update(detections, timestamp) -> list[Track]

```

ByteTrack is the V1 baseline. Current Ultralytics tracking documentation describes it as the simplest/fastest baseline and distinguishes it from BoT-SORT's optional ReID and camera-motion compensation. [?cite?turn7search1?](#)

16.2 Track buffer

The tracker retains lost tracks for a configurable number of frames before ending them. The exact buffer/timeout is deliberately validated against pilot footage because it directly affects re-entry and unique-count behavior. The PRD marks this as an open decision. [?filecite?turn9file7?L449-L458?](#)

16.3 No identity claim

A track ID means 'this sequence of detections is associated by the tracker.' It does not mean 'this is the same human forever.'

17. Counting and Crossing

```
bottom_center = ((x1+x2)/2, y2)
```

```
current_count =  
    number of ACTIVE tracks whose bottom_center is inside ROI
```

```
entry/exit:  
    compare previous bottom_center with current bottom_center  
    detect configured boundary crossing  
    accept at most one event per track per crossing state
```

The Master PRD specifically defines current count as active tracks inside the ROI and boundary crossing using the bottom-center/feet position, counted at most once per track per crossing. [?filecite?turn9file7?L449-L460?](#)

17.1 Crossing state

```
OUTSIDE -> APPROACHING -> CROSSING -> INSIDE  
INSIDE  -> APPROACHING_EXIT -> CROSSING -> OUTSIDE
```

The implementation should use a small hysteresis/debounce around the line so jitter at the boundary does not create multiple events.

18. Session and Approximate Unique Counting

A session starts when monitoring starts and ends on operator stop or application restart. Live count and active track state do not persist across restart; Core+ historical records may persist. [?filecite?turn9file7?L451-L458?](#)

```
session_seen_track_ids = set()
```

```
if track enters ROI for first time:  
    session_seen_track_ids.add(track.id)
```

```
unique_session_count = len(session_seen_track_ids)
```

A re-entering person whose old track expired may receive a new ID. This can inflate the approximate unique count. The dashboard must label it as approximate.

19. Occupancy and Crowd Analytics

```
if capacity is None or capacity <= 0:  
    occupancy = null  
    capacity_state = "NOT_SET"  
else:  
    occupancy = round((current_count / capacity) * 100)
```

```
crowd_level = threshold_map(occupancy)  
crowd_level = debounce(crowd_level)
```

The PRD explicitly resolves capacity-zero behavior as 'capacity not set' and requires configurable strictly increasing crowd thresholds plus debounce of the derived label. [?filecite?turn9file7?L463-L470?](#)

Input	Derived output
current_count	occupancy_percent
occupancy_percent	LOW/MODERATE/HIGH/CRITICAL
rolling measurements (Core+)	increasing/decreasing/stable trend

The PRD explicitly resolved the earlier ambiguity by requiring symmetric hysteresis: an alert must clear before it can re-fire.

[filecite?turn9file4?L291-L298?](#)

22. Live State Model

```
LiveState {
  schema_version
  timestamp
  session_id
  system_state
  camera_state
  performance
  vision
  counts
  occupancy
  crowd
  alerts
  freshness
}
```

Field	Meaning
timestamp	Time at which this state snapshot was produced
system_state	STARTING/LIVE/DEGRADED/UNSTABLE/OFFLINE/STOPPING
camera_state	CONNECTED/RECONNECTING/OFFLINE
performance	processing FPS, inference latency, frame age
vision	person/face/tracker health
counts	current and approximate unique session count
occupancy	capacity and calculated percentage
crowd	debounced crowd label
alerts	currently active P0 alerts
freshness	age of latest source frame/state

23. Backend/API

23.1 Boundary

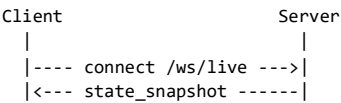
FastAPI is a communication layer, not a CV execution host. No endpoint performs YOLO inference synchronously for the request.



23.2 API responsibilities

- Read current state.
- Read/write validated configuration.
- Start/stop sessions.
- Read health.
- Read Core+ history/events.
- Expose WebSocket live state.

24. WebSocket




```

|<--- state_update -----|
|<--- alert_fired -----|
|<--- state_update -----|
|
|---- disconnect -----|
|
|---- reconnect ----->|
|<--- state_snapshot -----|

```

The first message after reconnect is a full state snapshot. Incremental updates are not replayed. This prevents the frontend from rebuilding state from a missing sequence.

FastAPI officially supports WebSockets and provides TestClient support for WebSocket tests. [?cite?turn2search11?turn1search3?](#)

25. Video Presentation Boundary

The low-volume WebSocket channel must not carry the full camera video stream. V1 should keep video presentation separate from metrics/state transport.

Need	V1 approach
Live metrics	WebSocket JSON
Configuration/history	REST JSON
Annotated video	Local browser-accessible stream/presentation mechanism only if required by the implemented UI; exact transport is isolated from metrics
Raw camera frames	Never sent through the metrics WebSocket

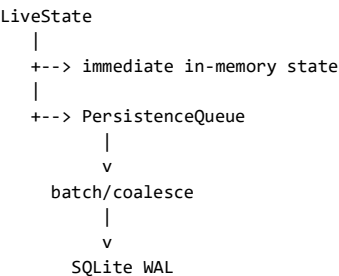
This separation prevents a slow browser or high-bandwidth video path from blocking low-volume operational state.

26. Database

26.1 Why SQLite

- One local deployment.
- Small number of tables.
- No remote clients required.
- Zero database server administration.
- Core+ history is aggregated rather than frame-level.
- WAL is sufficient for the local reader/writer pattern.

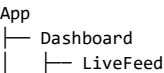
26.2 Persistence flow



The PRD explicitly says that camera/ROI configuration, Core+ measurements, alert events and system events do not require a database server in V1 and that raw video/snapshots are not stored by default. [?filecite?turn9file9?L547-L555?](#)

27. Frontend

27.1 Page model



- | | | CurrentCount
- | | | UniqueCountApprox
- | | | Occupancy
- | | | CrowdLevel
- | | | SystemState
- | | | CameraState
- | | | AlertBanner
- | | | HistoryChart (Core+)
- | | Configuration
- | Session / History

27.2 State ownership

State	Mechanism
Live operational state	WebSocket subscription + local live-state store
Configuration form	React component state; REST mutation
Historical measurements	TanStack Query
Connection/reconnect	WebSocket client state
Chart data	TanStack Query result transformed for Recharts

The dashboard must not read model names, confidence tensors, tracker IDs or CV implementation objects. It receives application-level state only.

28. Mock and Simulation

```
interface VisionStateSource {
  start()
  stop()
  current_state() -> LiveState
  subscribe(listener)
}
```

RealVisionStateSource
MockVisionStateSource

Scenario	Expected behavior
Normal	LIVE with changing counts
Crowd growth	Count/occupancy/crowd rise
Crowd reduction	Count/occupancy/crowd fall
Critical occupancy	P0 alert fires after debounce
Camera offline	OFFLINE + last reliable values
Detection degradation	DEGRADED/UNSTABLE depending on validated rules
Recovery	OFFLINE -> STARTING -> LIVE
WebSocket disconnect	CV continues; client reconnects and gets snapshot

This interface is the main mechanism that prevents heterogeneous developer hardware from becoming a project-wide bottleneck.

29. Data Contracts

29.1 Live state example

```
{
  "schema_version": 1,
  "timestamp": "2026-08-20T10:15:30.123Z",
  "session_id": "uuid",
  "system_state": "LIVE",
  "camera_state": "CONNECTED",
  "performance": {
    "processing_fps": 0.0,
    "inference_latency_ms": 0.0,
    "frame_age_ms": 0.0
  },
  "vision": {
    "person_detection": "OK",
    "face_detection": "OK",
    "tracking": "OK"
  },
}
```

```

"counts": {
  "current": 7,
  "unique_session_approx": 11
},
"occupancy": {
  "capacity": 10,
  "percent": 70
},
"crowd": {
  "level": "HIGH"
},
"alerts": []
}

```

Numeric performance values in this example are schema placeholders only and are not measurements.

30. API Contracts

Method	Endpoint	Use	Core/P0
GET	/api/v1/health	Overall application health	P0
GET	/api/v1/state	Complete current state snapshot	P0
GET	/api/v1/config	Read configuration	P0
PUT	/api/v1/config	Update ROI/capacity/threshold configuration	P0
POST	/api/v1/session/start	Start session	P0
POST	/api/v1/session/stop	Stop session	P0
GET	/api/v1/session/current	Current session metadata	P0
GET	/api/v1/health/camera	Camera diagnostics	P0
GET	/api/v1/health/vision	CV diagnostics	P0
GET	/api/v1/history	Aggregated history	Core+
GET	/api/v1/events	Persisted events	Core+
WS	/ws/live	Live state push	P0

30.1 Error envelope

```

{
  "error": {
    "code": "INVALID_CONFIGURATION",
    "message": "capacity must be greater than zero",
    "details": {}
  },
  "timestamp": "ISO-8601"
}

```

30.2 Contract rule

API schemas are versioned at `/api/v1`. Model names and tracker internals never appear in the public contract.

31. Database Schema

Table	Important fields	Indexes / constraints
sessions	id, started_at, ended_at, status	status enum; started_at index
configuration	id, source, roi_json, capacity, thresholds_json, updated_at	one active configuration
measurements	id, session_id, timestamp, count, occupancy_percent, crowd_level	session_id+timestamp index
events	id, session_id, timestamp, type, severity, fired_at, cleared_at, payload_json	session_id+timestamp; type+timestamp

```

sessions 1 —————< measurements
sessions 1 —————< events
configuration (active setup, independent of session)

```

31.1 No frame table

Do not create a detection-per-frame table. It would create unnecessary write volume and couple database growth to camera FPS. The live frame/detection/tracking data is transient unless it becomes a meaningful event or aggregated measurement.

31.2 WAL

WAL is enabled at startup and verified with `PRAGMA journal\_mode`. Backup procedures must treat the active WAL state as part of the database. SQLite documents that committed transactions can be lost if the database is separated from its WAL file during copying.

32. Repository Structure

```
visionqueue/
├─ pyproject.toml
├─ uv.lock / requirements lock equivalent
├─ README.md
├─ LICENSE / THIRD_PARTY_NOTICES.md
├─ config/
├─ models/
├─ scripts/
├─ backend/
│  └─ api/
│     └─ websocket/
│        └─ schemas/
│           └─ services/
├─ vision/
│  └─ camera/
│     └─ runtime/
│        └─ detection/
│           └─ face_detection/
│              └─ tracking/
│                 └─ frame_pipeline/
├─ domain/
│  └─ counting/
│     └─ occupancy/
│        └─ crowd/
│           └─ alerts/
│              └─ state/
├─ database/
│  └─ schema/
│     └─ repositories/
│        └─ migrations/
├─ frontend/
│  └─ src/
│     └─ components/
│        └─ pages/
│           └─ hooks/
│              └─ services/
│                 └─ state/
│                    └─ types/
├─ tests/
│  └─ unit/
│     └─ cv/
│        └─ integration/
│           └─ failure/
│              └─ performance/
│                 └─ fixtures/
```

The repository is organized around technical responsibilities and stable contracts, not around individual people or an equal number of modules.

33. Configuration

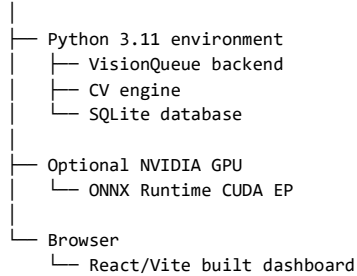
Configuration	Example type	Runtime effect
camera.source	device/file/URL	Camera input
camera.width/height	integer	Requested capture size
camera.target_fps	integer	Capture request where supported
vision.person_confidence	float	Detection filter
vision.detection_interval	integer	Frames between detector runs
tracking.track_buffer	integer	Lost-track lifetime
roi.x/y/width/height	integers	Counting zone
capacity	integer/null	Occupancy denominator

crowd.thresholds	percentages	Crowd labels
alerts.debounce_ms	integer	Alert fire/clear debounce
runtime.preferred_provider	CUDA/CPU	Inference provider preference
persistence.measurement_interval	seconds	Core+ history cadence

The exact numerical defaults remain validation parameters. They must be stored in configuration rather than scattered through code.

34. Deployment

Windows host



34.1 CPU-only development

A developer without a GPU can run mock mode, backend, database, analytics and frontend. CPU-only CV is supported for functional testing, with performance targets validated on the designated reference environment.

34.2 GPU environment

Only the CV runtime needs GPU-specific dependencies. The rest of the application remains identical.

34.3 Startup

49. Validate Python/runtime configuration.
50. Open database and enable/verify WAL.
51. Load configuration.
52. Initialize camera.
53. Load person and face models.
54. Select inference provider.
55. Initialize tracker and domain state.
56. Start FastAPI/WebSocket.
57. Enter STARTING.
58. Reach LIVE only after the first reliable reading.

35. Logging and Observability

Signal	Sampling/logging policy	Purpose
Processing FPS	Periodic metric	Performance state
Frame age	Periodic metric	Staleness detection
Inference latency	Periodic distribution	Model/runtime profiling
Camera status	On change	Failure/recovery
Provider	On startup/fallback	CUDA/CPU visibility
Track count	Periodic/debug	Tracker diagnostics
Detection confidence summary	Periodic	Reliability diagnostics
State transition	Every transition	Explain system behavior
Alert fire/clear	Every event	Auditability
DB write latency	Periodic/slow-write warning	Persistence isolation
WebSocket clients	On connect/disconnect	Live transport health

Do not log every frame at INFO level. High-frequency details belong at DEBUG or aggregate metrics.

36. Security and Privacy

- Bind the local API to localhost by default.
- Validate all configuration inputs.
- Do not store API keys in source control.
- Pin dependencies and retain a dependency/license inventory.
- Do not persist raw video by default.
- Do not persist face images or face embeddings.
- Track IDs are session-local technical identifiers.
- Display explicit camera/system status rather than hiding failures.

The PRD explicitly requires local processing, no biometric identification, no persistent identity and no raw video/snapshot retention by default. [filecite?turn9file9?L547-L563?](#)

37. Testing Strategy

37.1 Unit

- ROI membership.
- Bottom-center calculation.
- Boundary crossing.
- Crossing debounce.
- Current count.
- Unique-session count.
- Occupancy including capacity unset.
- Crowd threshold mapping.
- Crowd debounce.
- Alert hysteresis.
- Reliability state transitions.

37.2 CV validation

Scenario	Measure
Single person	Detection recall, track continuity, count correctness
Multiple people	Track continuity and count correctness
Crossing paths	ID switches and duplicate events
Partial occlusion	Track loss/recovery
Dense queue	Current count stability
Enter/exit	Crossing event correctness
Re-entry	Approximate unique-count behavior
Lighting changes	Detection confidence/recall
Different distances	Detection stability
Temporary detection loss	State transition and count freezing
Camera movement	Known limitation; confirm no silent false confidence

37.3 API/WebSocket

- REST schema validation.
- Invalid configuration.
- Session lifecycle.
- WebSocket connect.
- WebSocket reconnect.
- Snapshot after reconnect.
- Slow client handling.
- Backend restart behavior.

37.4 Frontend

- LIVE rendering.

- DEGRADED/UNSTABLE/OFFLINE states.
- Capacity not set.
- Alert fire/clear.
- WebSocket reconnect.
- Empty history.
- History chart data transformation.

37.5 End-to-end

The Definition of Done requires real video from the actual pilot queue, not only synthetic clips. [filecite?turn9file1?L64-L81?](#)

38. Performance Engineering

Metric	Target class	Measurement
Startup	PRD target: approximately 2 s to visible feed	Wall-clock measurement
Processing FPS	Usable real-time; exact value TBD	Sustained 5–10 min run
Frame age	Must remain bounded; exact threshold TBD	Capture timestamp vs processing timestamp
Inference latency	Stable under load	p50/p95/p99
End-to-end state latency	Low and stable; TBD	Frame capture to browser receipt
CPU/GPU utilization	No sustained saturation that causes instability	OS/GPU telemetry
Memory	No uncontrolled growth	30–60 min soak
DB write impact	No critical-path blocking	A/B persistence enabled/disabled

The PRD explicitly says FPS must be validated on the team's actual laptop and recommends frame skipping/tracking plus DEGRADED state if CPU performance is insufficient. [filecite?turn9file9?L577-L584?](#)

38.1 Performance experiment matrix

Variable	Values to test
Input resolution	at least two practical resolutions
Detection cadence	every frame vs every N frames
Provider	CPU vs CUDA where available
Face cadence	full vs reduced cadence
Persistence	disabled vs enabled
Browser clients	one vs multiple local clients

The best configuration is the one that meets the project's measured correctness and freshness requirements, not the one with the highest theoretical FPS.

39. Failure Injection and Recovery

Failure	Expected path
Camera unplugged	OFFLINE; preserve last reliable values; reconnect
Camera returns	STARTING; fresh confidence check; LIVE if healthy
CUDA provider failure	Record fallback; switch to CPU if feasible
CPU too slow	DEGRADED; reduce workload only through validated controls
Tracker instability	UNSTABLE; freeze counts
Database unavailable	Live CV continues; persistence error logged; no CV blocking
WebSocket client disconnect	CV continues; reconnect gets snapshot
Backend request failure	Return structured error; live loop unaffected
Stale frame backlog	Drop old frames; process newest
Browser tab closed	No impact on CV

The system must not interpret a database failure, WebSocket failure or browser failure as a camera failure.

40. Technical Risks

Risk	Likelihood	Impact	Mitigation	Validation
YOLO26n runtime/export incompatibility	Medium	High	Export ONNX early; run provider smoke test	Reference-machine smoke test
CUDA/cuDNN mismatch	Medium	High	Pin compatible ORT/CUDA/cuDNN	Startup provider test

			combination	
CPU cannot sustain real-time	Medium	High	Small model, lower resolution, frame skipping	Performance matrix
ID switches	Medium	High	Fixed camera, ByteTrack tuning, explicit limitation	Crossing/occlusion clips
Duplicate crossing	Medium	High	Bottom-center + crossing state + debounce	Boundary test suite
Face detector adds latency	Medium	Medium	Separate cadence; measure combined load	CV pipeline benchmark
Stale frames	Medium	High	Latest-frame buffer	Artificial slow-consumer test
DB blocks CV	Low	High	Persistence worker	Injected DB delay
Camera moved	Medium	Medium	Controlled pilot placement	Move-camera test; document limitation
20-day schedule	High	High	P0-first; cut Core+ if necessary	Milestone review
License incompatibility for future commercial use	Low V1 / high consequence future	High future	License inventory and review	Pre-commercial review

41. Architecture Rationale Records

AR-01 — Modular monolith

Context: one camera, one local host, short student build. Rationale: separate responsibilities without network/service overhead. Consequence: easiest debugging and deployment; future service boundaries can be introduced only when scale requires them.

AR-02 — ONNX Runtime instead of direct PyTorch inference

Context: need GPU acceleration plus CPU fallback and model-runtime separation. Rationale: ONNX is a deployment boundary and ORT exposes CPU/CUDA providers. Consequence: model export becomes part of the build/validation process.

[?cite?turn3search1?turn1search2?](#)

AR-03 — CUDA EP before TensorRT

Context: GPU is optional and the project needs maintainability. Rationale: one inference API with simpler fallback. TensorRT remains a measured optimization path only if CUDA fails the performance target.

AR-04 — ByteTrack before ReID

Context: fixed camera and no identity requirement. Rationale: lower complexity and overhead. Consequence: unique count remains approximate.

AR-05 — SQLite WAL

Context: local single-device history. Rationale: no server and sufficient local concurrency. Consequence: not suitable as the future centralized database.

AR-06 — WebSocket for state, not video

Context: low-volume changing metrics versus high-bandwidth frames. Rationale: separates operator state from video transport and protects the critical loop.

AR-07 — Mock contract

Context: heterogeneous hardware. Rationale: frontend/backend/database/analytics can progress without GPU CV.

42. Known Limitations

- ByteTrack IDs are not human identity.
- Re-entry after track timeout can create a new ID and inflate unique-session count.
- Heavy occlusion can cause ID switches.
- Camera movement can invalidate ROI; V1 does not automatically detect camera movement.

- Single camera cannot observe people fully hidden behind others.
- Phone-over-WiFi adds source latency and depends on network stability.
- Face detection quality is not equivalent to person-count accuracy.
- V1 does not provide reliable waiting-time estimation.
- V1 does not provide queue-jumping/abandonment/stagnation intelligence.
- CPU-only operation may be functionally correct but fail real-time performance targets on some hardware.

The PRD explicitly lists heavy-occlusion tracking, re-entry/new IDs, camera movement and single-camera operation as known limitations rather than hidden defects. [filecite?turn9file1?L73-L75](#)

43. Future / Advanced Architecture

Future stage	Possible capability	Dependency
V2	Improved tracking/counting, queue length, flow rate, multi-zone, density heatmaps	P0 tracking proven reliable
V3	Loitering, restricted areas, abnormal events, multi-camera/cross-camera	V2 tracking improvements
V4	Forecasting, wait-time prediction, staffing recommendations	Meaningful historical data
V5	AI operations assistant, natural-language analytics, voice, multi-location/cloud	Stable analytics API + budget

The PRD sequences the roadmap this way and explicitly says multi-camera should follow a proven single-camera system rather than force a redesign into V1. [filecite?turn9file4?L231-L252](#)

Potential future infrastructure such as PostgreSQL, Redis or cloud services is not part of the V1 runtime.

44. Validation Plan

Validation phase	Exit condition
Environment	All core dependencies install on reference Windows host
Inference	YOLO26n ONNX loads and runs with selected provider
Face	YuNet loads and produces visible detections
Tracking	Normal scenes maintain usable IDs
Counting	ROI and crossing unit tests pass
Reliability	All state transitions are observable and correct
Fallback	CUDA failure path reaches CPU or appropriate degraded state
Persistence	Core+ writes do not block CV
WebSocket	Reconnect produces fresh snapshot
Frontend	All reliability states render distinctly
Pilot	Actual queue footage produces measured results and known limitations are documented

45. Final Architecture Review

59. Every runtime component has one primary responsibility.
60. Frontend does not access CV internals.
61. Backend request handlers do not run heavy inference.
62. Database writes are outside the per-frame critical path.
63. GPU is isolated to the inference capability.
64. CPU-only development remains possible for non-CV work.
65. Mock state uses the same application-level contract as real state.
66. Current count and unique count are explicitly different.
67. Tracking IDs are not treated as identity.
68. Face detection is separate from counting and recognition.
69. Stale frames are discarded instead of accumulated.
70. Failure states preserve the last reliable value and expose uncertainty.
71. WebSocket carries low-volume state, not the video stream.
72. SQLite is limited to local aggregated persistence.

73. P0 is protected from Core+ scope.
74. Advanced architecture is not a V1 dependency.
75. No unnecessary broker, cache, cloud GPU, microservice or vector database is required.
76. All major performance claims remain measurement-based.

This review is intentionally aligned with the PRD's core product principle: prove one camera can become a live, trustworthy count and crowd-level reading, with uncertainty and failure exposed rather than hidden. [filecite?url=9file5%20L341-L346%20](#)

46. Technical Decisions Requiring Validation

77. Confirm exact Python 3.11 patch and package lock combination.
78. Confirm OpenCV 5.x + current YuNet model loading and face detection behavior.
79. Confirm YOLO26n ONNX export settings and output parsing.
80. Confirm ONNX Runtime 1.29.x CUDA package with the actual NVIDIA driver/CUDA/cuDNN environment.
81. Confirm CPU-only environment uses the CPU package rather than the GPU package.
82. Measure person detector accuracy on representative pilot footage.
83. Measure combined person + face pipeline performance.
84. Choose detection cadence based on freshness and track continuity.
85. Choose face-detection cadence based on college demo visibility versus latency.
86. Choose ByteTrack track buffer/timeout from real footage.
87. Choose person confidence threshold from labeled pilot clips.
88. Choose crowd threshold defaults with pilot site context.
89. Choose persistence measurement interval for Core+.
90. Validate that a slow WebSocket client cannot block state publication.
91. Validate database-delay injection cannot increase frame age.
92. Validate recovery transitions OFFLINE -> STARTING -> LIVE.
93. Confirm the exact video-presentation mechanism only if the final UI requires annotated live video beyond a local camera preview.

47. PRD ↔ TDD Consistency Check

PRD requirement	TDD coverage	Result
Single-camera local processing	System context + deployment	Aligned
Person + face detection	Separate person and face pipelines	Aligned
Person detection is counting signal	Detection/counting sections	Aligned
One ROI	Camera/config/counting	Aligned
Tracking IDs	ByteTrack + track lifecycle	Aligned
Approx. unique session count	Session registry + limitations	Aligned
Bottom-center crossing	Counting section	Aligned
Capacity unset handling	Occupancy section	Aligned
Crowd debounce	Analytics section	Aligned
Critical occupancy + camera/detection alerts	Alert engine	Aligned
STARTING/LIVE/DEGRADED/UNSTABLE/OFFLINE	Reliability state machine	Aligned
Core+ trend/peak/history	Database + analytics, conditional	Aligned
No raw video/snapshot retention by default	Database/privacy	Aligned
P2 waiting time/queue movement	Explicitly deferred	Aligned
Future multi-camera/cloud/assistant	Future section only	Aligned
Track timeout	Explicit validation item	Open by PRD
Crowd thresholds	Explicit validation item	Open by PRD
GPU availability	Hardware-neutral; provider capability	Open by PRD
Pilot site	Not hard-coded; validation context only	Open by PRD

The PRD itself marks the pilot site, GPU availability, remaining schedule, track timeout and crowd thresholds as open questions. The TDD does not silently invent those values. [filecite?url=9file1%20L470-L475%20](#)

47.1 Remaining ambiguity

- The exact transport for an annotated browser-visible live video feed is not specified in the PRD. The TDD deliberately keeps it as a separate boundary rather than inventing a transport technology.
- The exact face-detection cadence is not specified; it is treated as a performance implementation parameter.

- The exact API endpoint names are a technical design choice and therefore are not a PRD conflict.

48. References

Source	Reference
Master PRD	VisionQueue_Master_PRD-1.pdf — authoritative product specification supplied in the project files.
Ultralytics YOLO26	Official YOLO26 model/export/licensing documentation — https://docs.ultralytics.com/models/yolo26
Ultralytics Export	Official YOLO export/ONNX documentation — https://docs.ultralytics.com/modes/export
Ultralytics Tracking	Official tracker documentation — https://docs.ultralytics.com/modes/track
Ultralytics licensing	Official licensing information — https://docs.ultralytics.com/
ONNX Runtime	Official Python installation — https://onnxruntime.ai/docs/get-started/with-python.html
ONNX Runtime CUDA	Official CUDA Execution Provider compatibility — https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html
OpenCV / YuNet	OpenCV Zoo YuNet — https://github.com/opencv/opencv_zoo/tree/main/models/face_detection_yunet
ByteTrack	FoundationVision ByteTrack — https://github.com/FoundationVision/ByteTrack
FastAPI	Official features/WebSocket/testing documentation — https://fastapi.tiangolo.com/
SQLite WAL	Official SQLite WAL documentation — https://sqlite.org/wal.html
React	Official React docs/version — https://react.dev/
Vite	Official Vite guide — https://vite.dev/guide/
Tailwind CSS	Official Tailwind Vite integration — https://tailwindcss.com/docs/installation/using-vite
Recharts	Official Recharts documentation — https://recharts.github.io/
TanStack Query	Official TanStack Query React docs — https://tanstack.com/query/latest/docs/framework/react/quick-start
TypeScript	Official TypeScript 5.9 docs — https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-9.html

Important technical claims in this document are based primarily on official documentation and original/open-source project sources. Community opinions are not used as authoritative evidence.